Assignment-11.2

Task 1 – Stack Using AI Guidance

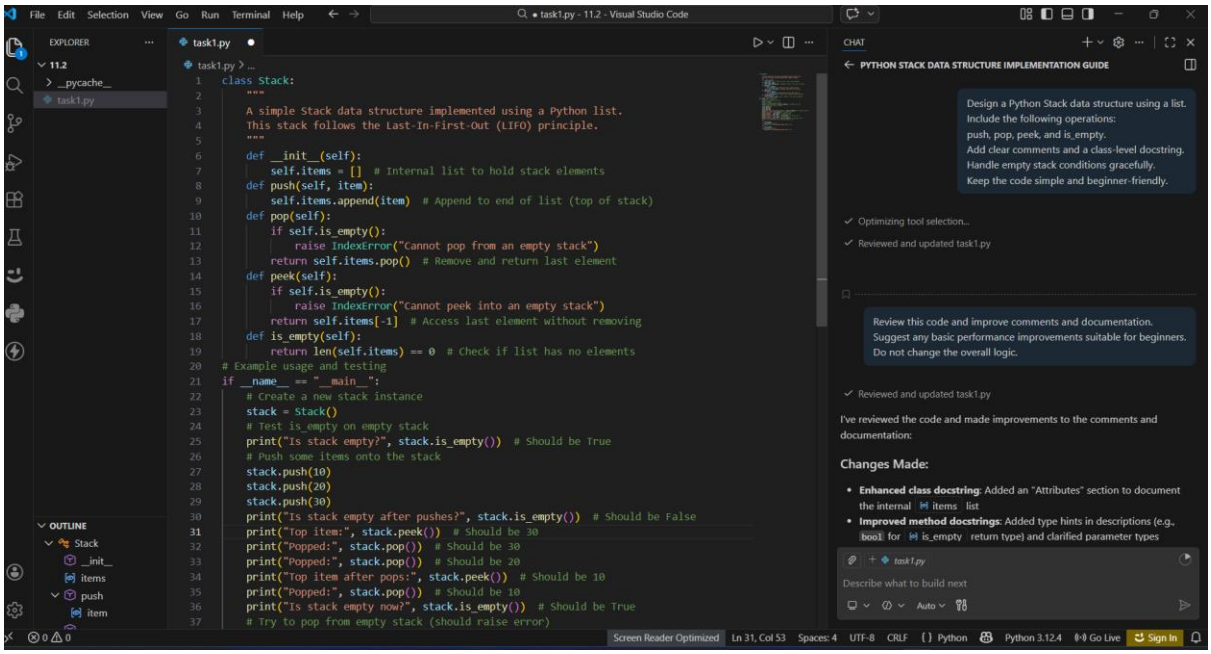
Objective

To design and implement a Stack data structure in Python using AI assistance, supporting basic stack operations such as push, pop, peek, and empty check.

AI Prompt Used

Design a Python Stack data structure using a list.
Include the following operations:
push, pop, peek, and is_empty.
Add clear comments and a class-level docstring.
Handle empty stack conditions gracefully.
Keep the code simple and beginner-friendly.

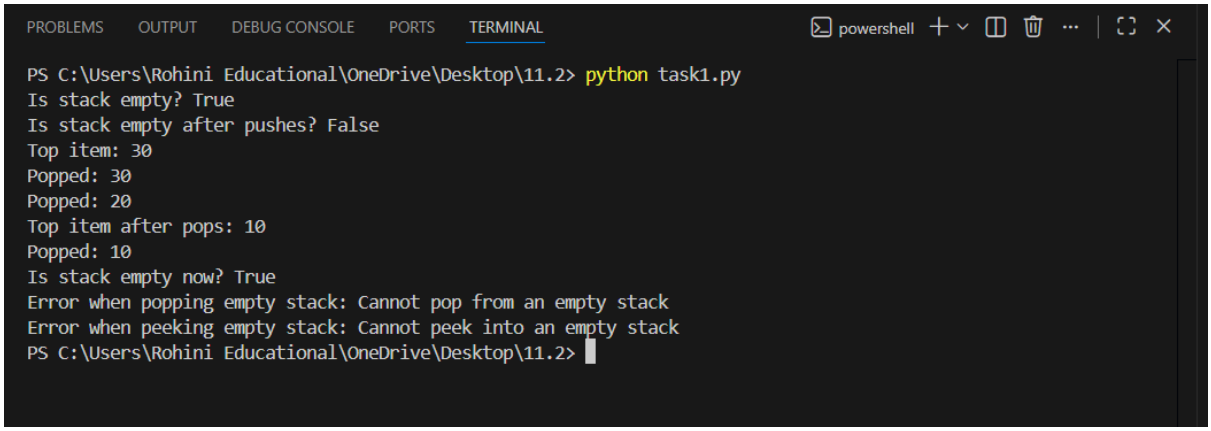
AI-Generated Code



Explanation

- Stack follows the Last In First Out (LIFO) principle.
- Python list is used to store stack elements.
- push() adds an element to the stack.
- pop() removes the top element and raises an error if the stack is empty.
- peek() returns the top element without removing it.

OUTPUT



Result

The Stack data structure was successfully implemented and executed in VS Code using AI guidance.

Task 2 – Queue Design Using AI Guidance

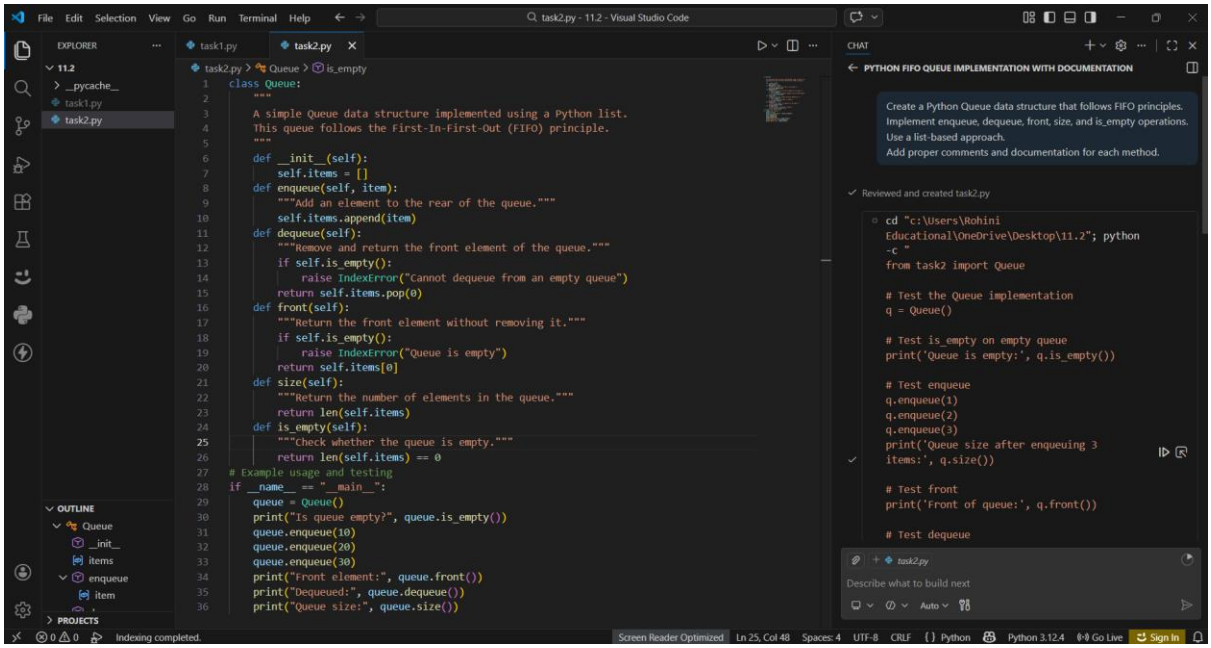
Objective

To design and implement a Queue data structure in Python using AI assistance, following the First-In-First-Out (FIFO) principle and supporting basic queue operations.

AI Prompt Used

Create a Python Queue data structure that follows FIFO principles.
Implement enqueue, dequeue, front, size, and is_empty operations.
Use a list-based approach.
Add proper comments and documentation for each method.

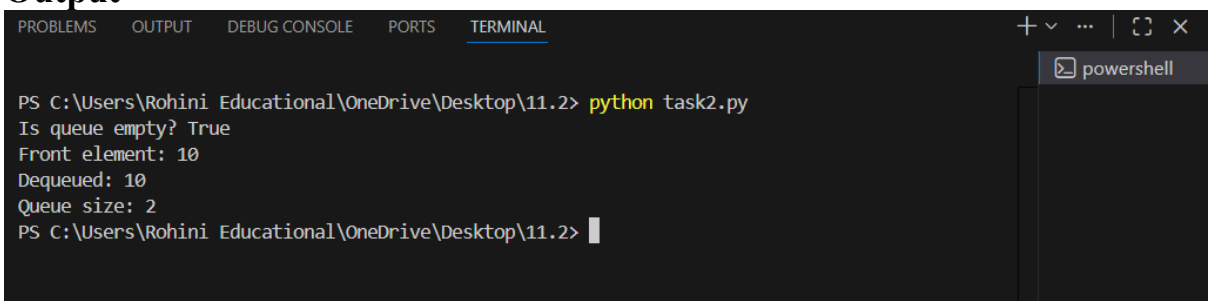
AI-Generated Code



Explanation

- Queue follows the FIFO (First In First Out) principle.
- Python list is used to store queue elements.
- enqueue() adds elements to the rear of the queue.
- dequeue() removes the front element and raises an error if the queue is empty.
- front() returns the first element without removing it.

Output



Result

The Queue data structure was successfully implemented and executed in VS Code using AI guidance.

Task 3 – Singly Linked List Construction Using AI Guidance

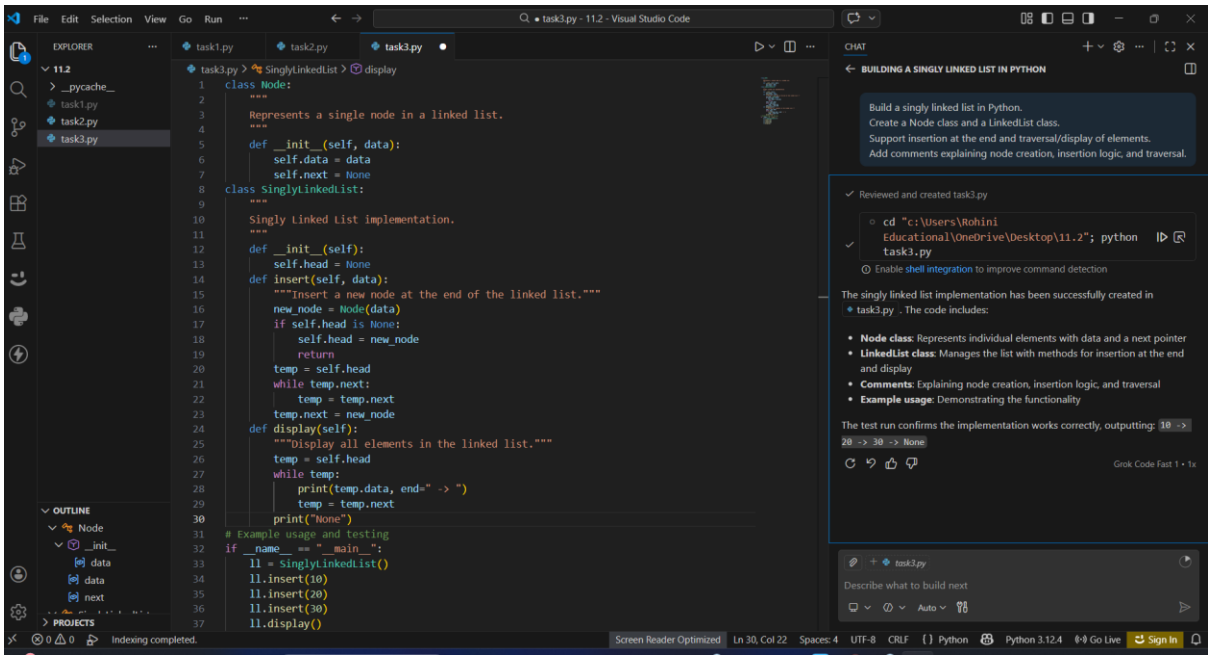
Objective

To design and implement a Singly Linked List in Python with AI assistance, supporting node creation, insertion, and traversal.

AI Prompt Used

Build a singly linked list in Python.
Create a Node class and a LinkedList class.
Support insertion at the end and traversal/display of elements.
Add comments explaining node creation, insertion logic, and traversal.

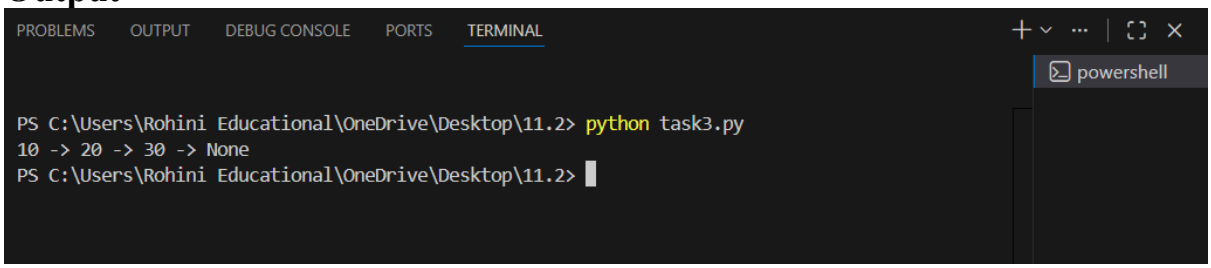
AI-Generated Code



Explanation

- A linked list is made up of nodes connected by pointers.
- Each node contains data and a reference to the next node.
- insert() adds a new node at the end of the list.
- display() traverses the list and prints all elements.
- This implementation uses dynamic memory allocation.

Output



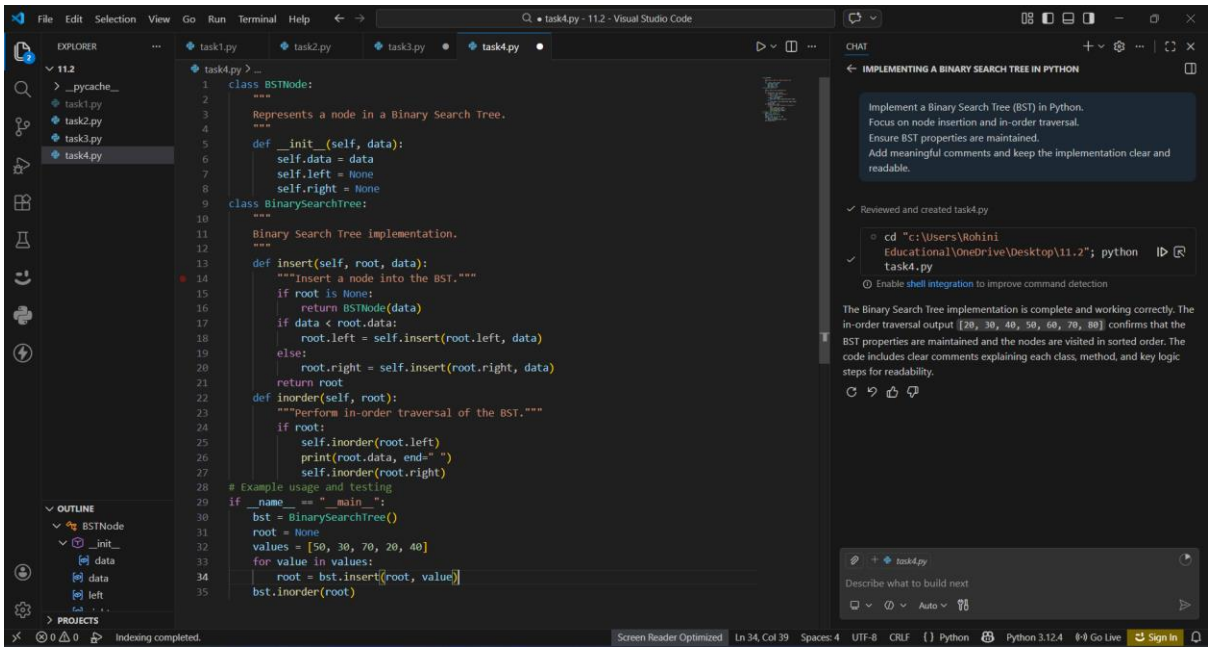
Result

The Singly Linked List was successfully implemented and executed in VS Code using AI guidance.

Task 4 – Binary Search Tree Operations Using AI Guidance

Objective
To design and implement a Binary Search Tree (BST) in Python using AI assistance, focusing on node insertion and in-order traversal.

AI Prompt Used
Implement a Binary Search Tree (BST) in Python.
Focus on node insertion and in-order traversal.
Ensure BST properties are maintained.
Add meaningful comments and keep the implementation clear and readable.

AI-Generated Code


Explanation
- A binary search tree maintains sorted data.
- Left subtree contains values smaller than the root.
- Right subtree contains values greater than the root.
- insert() places nodes according to BST rules.
- In-order traversal prints elements in ascending order.

Output


Result
The Binary Search Tree was successfully implemented and executed in VS Code using AI guidance.

Task 5 – Hash Table Implementation Using AI Guidance

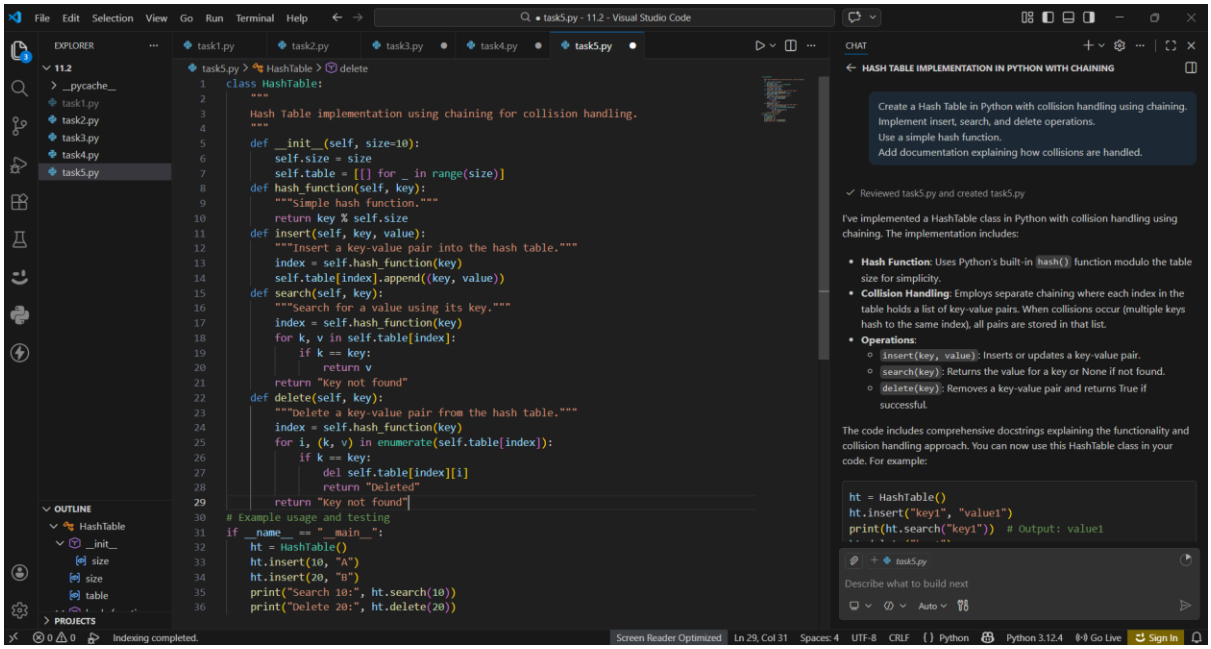
Objective

To design and implement a Hash Table in Python with AI assistance, supporting insert, search, and delete operations, with collision handling via chaining.

AI Prompt Used

Create a Hash Table in Python with collision handling using chaining.
Implement insert, search, and delete operations.
Use a simple hash function.
Add documentation explaining how collisions are handled**.**

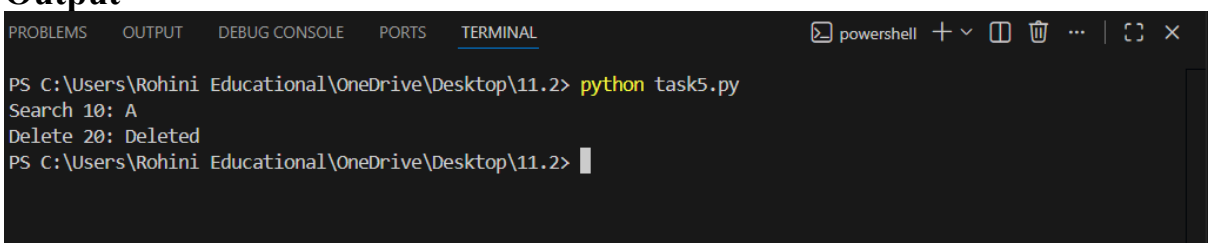
AI-Generated Code



Explanation

- Hash Table stores data in key-value pairs.
- A hash function maps keys to table indices.
- Chaining is used to handle collisions by storing multiple elements at the same index.
- insert() adds data to the table.
- search() retrieves values using keys.
- delete() removes key-value pairs safely.

Output



Result

The Hash Table was successfully implemented and executed in VS Code using AI guidance.